

CourseForge

Algorithm Analysis and Course Planning System

Course Project Report

Student: Yosef Asad

Student ID: 202417427

Course: Algorithms Analysis and Design

Instructor: Dr. Rasheed Yousef

University: Arab American University

August 2026

Abstract

This project is about a small program called CourseForge, for course planning. Implemented to use basic algorithmic skills of Chapter 1 - 4. This system produces a deterministic dataset that is based on a student-specific seed and provides five main functions: sorting student records, producing a valid study order according to course pre-requisites, choosing the best course Avoiding credit limit, binary searching, and computing the sum of the digits of a number. Greatest common divisor; Least common multiple.

The project sorts the elements using Selection Sort, Insertion Sort, topological sorting. The methods employed for the problem are: Source Removal, Exhaustive subset search, Iterative Binary Search, and Euclid's algorithm. Theoretically, four algorithms are analysed. experimentally. The experimental analysis measures operation of the algorithms. for increasing input sizes and compares the observed behaviour with the predicted efficiency classes.

Contents

1	Introduction	3
2	Dataset Generation	3
2.1	Seed	3
2.2	Generated Courses	3
2.3	Generated Student Records	4
3	Algorithm Implementation	4
3.1	Selection Sort	4
3.2	Insertion Sort	4
3.3	Topological Sort	5
3.4	Exhaustive Subset Search	5
3.5	Binary Search	6
3.6	Euclid's Algorithm	7
4	Theoretical Analysis	7
4.1	Selection Sort Derivation	8
4.2	Insertion Sort Derivation	8
4.3	Topological Sort Derivation	9
4.4	Binary Search Derivation	9
5	Experimental Analysis	9
5.1	Sorting Algorithms	9
5.2	Binary Search	10
5.3	Discussion	10
6	AI Assistance and References	11
7	Conclusion	11
8	References	12

1 Introduction

CourseForge is an application containing 5 Modules each with it's own application and algorithms, picked from a menu by the user depending on his needs. Before we get to the modules we should start with the dataset generation and dataset itself that are used by the 5 modules.

CourseForge requires a dataset of courses, student records and a random generator for that data to be made using a specific seed that is student dependent (Student ID). It goes through the modules to sort using different algorithms by GPA, order based on prerequisites, search for the best course load, binary search, Euclid's GCD & LCD and experimental algorithm analysis.

2 Dataset Generation

2.1 Seed

The dataset is generated using the following student-specific seed:

$$202417427 \bmod 10000 = 7427$$

The seed determines the dataset parameters:

$$n = 12 + (7427 \bmod 6) = 17$$

$$m = 40 + (7427 \bmod 20) = 47$$

$$C = 15 + (7427 \bmod 4) = 18$$

where:

- n is the number of courses,
- m is the number of student records,
- C is the credit cap.

2.2 Generated Courses

The course class consists of a couple fields:

- course code: The ID for the course, starting with CS for Computer Science as shown in

the project explanation.

- course name: The name of the course denoting the covered topics.
- credit hours: the value of the course in hour credits.
- value: the relative desirability or benefit of selecting a course.
- prerequisite: The courses required to take before this course.

2.3 Generated Student Records

Meanwhile the student record fields:

- student ID: The unique code ID for the student.
- student name: The name of the student.
- GPA: The current GPA rate of the student.

3 Algorithm Implementation

3.1 Selection Sort

Selection works by selecting the smallest next element through comparison to swap with the current element. In our specific case We were tasked to sort by studnets' GPA which was easily extracted from the implemented Record class.

Selection sort performance meters:

- GPA comparisons:

$$\frac{n(n-1)}{2} = \frac{47(46)}{2} = 1081$$

- swaps =

$\Theta(n)$ average, in our case: 41.

3.2 Insertion Sort

Insertion traces similarly as bubble sort but doesn't really compare like it, it assumes the array to be divided into 2 parts, sorted and unsorted. It compares the first element of the unsorted segment with the entire sorted segment until it's not less than compared element in the sorted segment to find the insertion position.

State the measured operations:

- GPA comparisons:

$$\frac{n^2}{4} = \frac{47^2}{4} = 552 \text{ average, in our case } 539$$

- shifts:

$$\frac{n^2}{4} \text{ average, in our case: } 494$$

3.3 Topological Sort

The prerequisites study was implemented using Topological sort because Directed Acyclic Graphs (DAG) are perfect for visualizing courses and their prerequisites, since it's a linear ordering of vertices for every directed edge such as $u \rightarrow v$, vertex u comes before v in the ordering.

The algorithm starts by putting all courses into a queue that have no courses remaining as pre-requisites. The course is then taken of the queue and placed in the study order, and the number of courses that depend on it is decreased by one. If a dependent course has all its prerequisites fulfilled, it is placed on the queue for processing, and will then be added to the queue. This process is repeated until the queue is empty. Lastly, if the total number of courses is more than the number of courses processed, then the resulting order is a valid study order; otherwise, if the total number of courses is less than the number of courses processed, a cycle is present in the pre-requisite graph.

3.4 Exhaustive Subset Search

Exhaustive subset search was used to implement the best course load problem. This module's eligible courses will be the first 10 courses in the generated course list. The algorithm iterates over all possible subsets of the set of eligible courses, instead of using a more sophisticated optimization algorithm to try to find the best combination.

Since every course can either be selected or not selected, the total number of possible subsets is:

$$2^k = 2^{10} = 1024$$

An image with a binary mask is used to represent each subset. For each of the subsets, the algorithm tests the 10 available courses to see if the corresponding bit is on. If it is established, then the number of credit hours and value assigned to the course will be added to the existing totals for that course.

The subset is only considered if the sum of the credits in the subset is below the credit limit, which has been calculated above:

$$\text{credits} \leq C$$

The final course load is thus the optimum selection of courses that can be chosen under the credit limit.

This is an exponential growth since every time a new course appears in the list, the number of subsets doubles. The number of subsets is:

$$O(2^k)$$

For the required $k = 10$, this results in only 1024 subsets, which is small enough to be checked directly.

3.5 Binary Search

Since the student's GPA was known to be sorted in ascending order, Binary search was used to search for the student's GPA. This algorithm makes the assumption that the records are already sorted as it will eliminate half the search range after every compare.

The algorithm begins with two indexes, *low* and *high* that indicate the range being searched. The middle position is found by calculating:

$$mid = low + \frac{high - low}{2}$$

This process continues until the target is found or the search range becomes empty. The implementation also counts each GPA comparison performed by the algorithm.

Since the search range is approximately divided in half after every comparison, the number of comparisons grows logarithmically with the input size. The worst-case number of comparisons is:

$$C(n) = \lfloor \log_2 n \rfloor + 1$$

Therefore, the efficiency class of Binary Search is:

$$O(\log n)$$

3.6 Euclid's Algorithm

An implementation of euclid's algorithm was made to find out the greatest common divisor of two integers. The algorithm continues to perform the modulo operation, and it will keep going until the second value is 0.

The remainder is the number less the multiple of the divisor, for two numbers a and b it is:

$$r = a \bmod b$$

Then the values are updated so that:

$$a = b, \quad b = r$$

The steps are repeated until $b \neq 0$. The value of a is the greatest common factor of the two numbers in the beginning of the loop. The implementation also figures how many times a modulo operation is used.

Once the GCD is found, the least common multiple (LCM) is found using:

$$\frac{a \cdot b}{\gcd(a, b)}$$

Repeatedly checking possible divisors is much less efficient than Euclid's algorithm. It has a logarithmic running time when applied to the values of the input numbers, typically written as:

$$O(\log(\min(a, b)))$$

The number of modulo operations varies with the input sizes, but the algorithm is efficient even for moderately sized integers.

4 Theoretical Analysis

The following four algorithms were selected for theoretical analysis:

1. Selection Sort,
2. Insertion Sort,
3. Topological Sort,
4. Binary Search.

Table 1: Theoretical analysis of selected algorithms

Algorithm	Input Size	Basic Operation	Efficiency Class
Selection Sort	n	GPA comparison	$\Theta(n^2)$
Insertion Sort	n	GPA comparison	$O(n^2)$ worst case
Topological Sort	n vertices, e edges	Edge processing	$O(n + e)$
Binary Search	n	Key comparison	$O(\log n)$

4.1 Selection Sort Derivation

Show the summation:

$$C(n) = (n - 1) + (n - 2) + \cdots + 1$$

Therefore:

$$C(n) = \frac{n(n - 1)}{2}$$

Thus Selection Sort has quadratic growth:

$$\Theta(n^2)$$

4.2 Insertion Sort Derivation

Discuss best and worst cases.

Worst case:

$$C(n) = 1 + 2 + \cdots + (n - 1)$$

Therefore:

$$C(n) = \frac{n(n - 1)}{2}$$

Thus the worst-case efficiency is:

$$O(n^2)$$

4.3 Topological Sort Derivation

Each course is processed once and each prerequisite edge is processed once.

Therefore:

$$C(n, e) = O(n + e)$$

4.4 Binary Search Derivation

Each comparison approximately halves the remaining search range.

Therefore the worst-case number of comparisons is:

$$C(n) = \lfloor \log_2 n \rfloor + 1$$

Thus Binary Search has logarithmic efficiency:

$$O(\log n)$$

5 Experimental Analysis

The algorithms were tested using five input sizes:

$$100, 200, 400, 800, 1600$$

Each experiment was executed three times and the results were averaged.

5.1 Sorting Algorithms

Selection Sort and Insertion Sort were measured using the number of key comparisons.

Table 2: Sorting experiment results

n	Selection Sort	Insertion Sort
100	4950	2574
200	19900	10149
400	79800	40299
800	319600	160399
1600	1279200	640799

5.2 Binary Search

Binary Search was tested using the same five input sizes.

Table 3: Binary Search experiment results

n	Average Comparisons	Ratio
100	7	–
200	8	1.14
400	9	1.13
800	10	1.11
1600	11	1.10

5.3 Discussion

The behavior of the algorithms in the experiments is consistent with the theory. Selection Sort can be easily seen as having a quadratic growth pattern. The number of comparisons is about four times as large if the input size is doubled. This is because the records are always compared the same number of times in Selection Sort because the comparisons are always in the same order:

$$\frac{n(n-1)}{2}$$

For instance, when $n = 200$ the number of comparisons rises to 19900 (roughly four times as many), whereas for $n = 100$ it was 4950.

The measured results also indicate quadratic behaviour of Insertion Sort. It is not like Selection Sort where the actual number of comparisons depends on the initial order, it does have a best case scenario of $n-1$ comparisons and a worst case scenario of $n(n-1)/2$ comparisons. The more the records are already sorted, the fewer the number of shifts and comparisons necessary, and a very poorly ordered input may require a great deal of work. This is why the measured values are not the same as Selection Sort, although both algorithms have a worst time of $O(n^2)$.

The sorting algorithms are quite different from the Binary Search. The number of comparisons only increases by about one if you double the size of the input. The measured results go up from 7 comparisons at $n = 100$ to 11 comparisons at $n = 1600$. The efficiency of this is in accordance with the logarithmic efficiency of Binary Search, where about half of the search space is eliminated each time.

Overall, the experimental results agree with the anticipated efficiency classes. The sorting algorithms scale quadratically and Binary Search scales logarithmically. The measurements of the

machine thus give practical proof for the theoretical analysis given above.

6 AI Assistance and References

This project has only used AI as a supporting tool. The use of AI was primarily for support in implementing the random data generator and for enhancing spacing and formatting within the source code, to make it easier to read.

The algorithms and their implementations were otherwise researched and developed based on the references listed at the end of this report. Algorithmic Technique like Selection Sort, Insertion Sort, Topological Sort, exhaustive subset search, binary search and euclid's algorithm was studied from the course material and the referred material.

The project work was completed by implementing the code, selecting the algorithms, counting the number of operations, conducting an experimental analysis and interpreting the results. The AI assistance was not used to replace the study and analysis of the algorithms but to use as an aid to programming in the areas mentioned above.

7 Conclusion

CourseForge implements the five modules using the indicated algorithms that it successfully implements. The system has been designed to generate a deterministic dataset by using the student specific seed, and uses this dataset to sort the student records, generate a legal study order, determine optimum course load, execute Binary Search and to calculate GCD and LCM.

The implementation also shows the disparity between theoretical and practical analysis of algorithms. The selection sort and insertion sort algorithms have quadratic growth on an increase in the input size, whereas the binary search algorithm has logarithmic growth. The numbers of operations measured for the three tests are as expected according to the theoretical results: SelectionSort performs a constant number of quadratic comparisons, while InsertionSort has a performance that is dependent on the ordering of the input.

The project also serves to illustrate the use of different algorithmic techniques in different types of problems. Topological Sort is applicable to course prerequisites, exhaustive search is applicable to the small number of courses that are eligible in the course-load problem, Binary Search offers an efficient way of searching sorted records, and Euclid's algorithm is applicable to the GCD problem.

Overall, the project was a practical experience in implementing algorithms without using built-in sorting or searching methods, estimating the running time of basic operations, and comparing the running time of the algorithms with the theoretical efficiency classes.

8 References

1. A. Levitin, *Introduction to the Design and Analysis of Algorithms*, Chapters 1–4.
2. Oracle, *Java Documentation*. Available at: <https://docs.oracle.com/en/java/>
3. GeeksforGeeks, “Selection Sort Algorithm.” Available at: <https://www.geeksforgeeks.org/dsa/selection-sort-algorithm-2/>
4. GeeksforGeeks, “Insertion Sort Algorithm.” Available at: <https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/>
5. GeeksforGeeks, “Topological Sorting Using BFS – Kahn’s Algorithm.” Available at: <https://www.geeksforgeeks.org/dsa/topological-sorting-indegree-based-solution/>
6. Stack Overflow, “Efficient Algorithm to Check for Subset Against a Range of Sets.” Available at: <https://stackoverflow.com/questions/34853095/efficient-algorithm-to-ch>
7. GeeksforGeeks, “Binary Search in Java.” Available at: <https://www.geeksforgeeks.org/java/binary-search-in-java/>
8. Stack Overflow, “How to Use Euclid’s Algorithm to Find GCF/GCD.” Available at: <https://stackoverflow.com/questions/41732993/how-to-use-euclids-algorithm-to-fin>